Malware Images Revisited: Reproducing GIST + k-NN Under Duplicate-Aware Splits, a Second Dataset, and Append Attacks

Theodore DellaVilla
Department of Cybersecurity, Rochester Institute of Technology
Rochester, NY, USA
td5360@rit.edu

***Abstract*—Visualizing a malware binary as a grayscale "byteplot" and classifying its texture is a static, execution-free approach whose reported accuracy on the Malimg benchmark has sat near 98–99% since Nataraj et al. introduced it in 2011. We reproduce their GIST + k-NN classifier from scratch and test whether that accuracy survives three changes to the evaluation: a split that keeps duplicate and near-duplicate images on one side, a second dataset (Microsoft BIG 2015), and functionality-preserving byte appending. The reproduction succeeds: GIST + k-NN reaches 97.46 $\pm$ 0.04% accuracy on Malimg against $\approx$98% in the original, but its macro-F1 is only 93.56 $\pm$ 0.09%, and a 256-bin byte histogram with logistic regression comes within a point of it. We find that 800 Malimg images (one family) are pixel-identical; under a duplicate-aware split, GIST + k-NN falls to 94.95 $\pm$ 0.05% on the families that can still be tested, while a fine-tuned ResNet-50 holds 99.03 $\pm$ 0.10%. Appending zero bytes equal to 1% of a file's size drops GIST + k-NN from 94.95% to 73.79% (21.23% at 10%), whereas the CNN stays above 93.85% up to a 5% budget. Malimg accuracy alone therefore says little about how well a byteplot classifier generalizes or resists evasion.**

***Index Terms*—malware classification, byteplot, GIST, reproducibility, data leakage, adversarial robustness**

I. Introduction

Static malware classification assigns a sample to a family without running it. This avoids the cost and risk of dynamic analysis and scales to the volume of samples that security vendors receive each day. In 2011, Nataraj et al. [1] proposed a strikingly simple static method: read the binary as a sequence of bytes, lay the bytes out as a grayscale image (a *byteplot*), describe the image's texture with the GIST descriptor [2], and classify it with k-nearest neighbours. On a new dataset of 25 families, Malimg, they reported about 98% accuracy. Over the following decade, convolutional neural networks (CNNs) pushed the same benchmark to 98–99% [3]–[6].

Such uniformly high numbers raise a question that the literature rarely asks: are they a property of the methods, or of the benchmark and how it is evaluated? Work on experimental bias in security machine learning [7], [8] shows that sampling choices, duplicated data and unrealistic threat models can inflate results substantially. Malimg is an attractive case study because the original method is cheap, fully described in the image domain, and still widely used as a baseline.

This paper reproduces [1] from scratch and extends it with a duplicate-aware split, a second dataset and an evasion study. It answers four research questions:

- **RQ1 (Reproduction):** Does GIST + k-NN reach roughly 98% on Malimg under the protocol of [1], and how close does a fine-tuned CNN come to [3], [5] under the same split?
- **RQ2 (Leakage):** How much accuracy survives when exact and near-duplicate images are kept on one side of the split?
- **RQ3 (Generalization):** Do the method rankings hold on Microsoft BIG 2015 under an identical pipeline?
- **RQ4 (Robustness):** How quickly do GIST + k-NN and the CNN degrade under functionality-preserving byte appending?

Our contributions are: (i) an independent, from-scratch reproduction of GIST + k-NN with every unstated parameter listed as an explicit assumption (Table III); (ii) a duplicate analysis of Malimg showing that one family consists of a single image repeated 800 times, and a grouped evaluation that separates leakage from untestable families; (iii) the same pipeline applied unchanged to BIG 2015; (iv) an append-attack study comparing GIST + k-NN, a fine-tuned ResNet-50 and a byte-histogram baseline under black-box and white-box attacks; and (v) a public repository in which one command regenerates every number in this paper. Unlike recent adversarial work on Malimg [9], which builds a stronger detector, we ask how far the original method's reported accuracy can be trusted.

II. Background

Byteplots. A byteplot reads a file as a vector of 8-bit unsigned integers and reshapes it into a two-dimensional grayscale image, one pixel per byte. Nataraj et al. fix the image width by file size, from 32 pixels for files under 10 KB to 1,024 pixels above 1,000 KB, and let the height follow from the file length. Sections of a Portable Executable (PE) file then appear as visually distinct bands: code is fine-grained texture, zero padding is black, and data or resources form their own patterns. Variants of one family, which typically share code, produce similar textures.

GIST. GIST [2] summarizes the spatial layout of an image's frequency content. The image is resized, normalized, and filtered with a bank of Gabor filters at several scales and orientations; the magnitude of each filter's response is averaged over a coarse $M \times M$ grid. With N filters, the descriptor has M^2N values. Nataraj et al. use $N = 20$ subbands and $M = 4$, giving 320 features, and classify with k-NN under Euclidean distance.

CNN transfer learning. Residual networks [10] pretrained on ImageNet can be fine-tuned on a small dataset by replacing the final layer. Applied to byteplots, the network learns texture features directly instead of using a hand-designed descriptor.

Appending to a PE file. Bytes appended after the last section of a PE file (the *overlay*) are not mapped into memory by the loader, so they do not change the program's behaviour [11]. Appending is therefore the simplest functionality-preserving manipulation available to an attacker, and it changes the byteplot: the image grows taller and may move into a wider row of the width table.

III. Related Work

Image construction. Nataraj et al. [1] introduced byteplots and Malimg. A companion study [12] compared binary texture analysis with dynamic analysis and argued that texture features reach comparable accuracy at a fraction of the cost. Ni et al. [13] built images from SimHash encodings of opcode sequences rather than raw bytes, and Vasan et al. [5] used color instead of grayscale images. A recent systematization of knowledge [14] organizes visualization-based detection by pipeline stage; we follow its structure here.

Features and classifiers. After GIST, most work replaced hand-designed descriptors with CNNs. Kalash et al. [3] fine-tuned a VGG16-based network (M-CNN) and reported 98.52% on Malimg and 98.99% on BIG 2015. Gibert et al. [4] trained a CNN from scratch on both datasets (98.48% on Malimg, 97.5% on BIG 2015, 10-fold CV). Vasan et al. fine-tuned CNNs on color byteplots (IMCFN, 98.82% on Malimg) [5] and combined several architectures in an ensemble (IMCEC) [6]. MalConv [15] skips the image altogether and applies a CNN to the raw byte sequence. Table I lists these results with the details that decide whether they are comparable to ours.

Evaluation practice. TESSERACT [7] showed that spatial and temporal bias in how data are split can inflate malware classifiers' reported performance far above what they achieve in deployment. Arp et al. [8] catalogued common pitfalls in security machine learning, including sampling bias, data snooping, inappropriate baselines and lab-only evaluation. Neither study examines Malimg's duplicate structure, which is the focus of our RQ2.

Robustness. Kolosnjaji et al. [16] showed that appending a few gradient-chosen bytes evades MalConv, and Demetrio et al. [11] surveyed practical functionality-preserving attacks on Windows malware detectors, from appending to header and section manipulation. AMGmal [17] uses adaptive masks to minimize the perturbation needed to evade a detector. Tariq et al. [9], using the same 9,339-image version of Malimg as we do, balance the families with SVM-SMOTE, generate FGSM perturbations restricted by a PE-structure mask to slack and writable regions, and retrain on the adversarial samples. They report an 81% evasion rate while modifying 0.284% of bytes, with retraining restoring accuracy from 16% to 97%. Their evaluation uses a single random 80:20 split without deduplication, the setting our RQ2 examines. To our knowledge, no prior work measures how the original GIST + k-NN pipeline responds to append attacks.

IV. Methodology

We run four experiments. E1 reproduces [1] on Malimg with a stratified split; E2 repeats it with a duplicate-aware grouped split; E3 applies the E2 protocol to BIG 2015; and E4 attacks the E2 models with appended bytes. Each experiment evaluates three models: GIST + k-NN (the reproduction), a fine-tuned ResNet-50 (a strong CNN baseline) and a byte-histogram logistic regression (a deliberately weak baseline that ignores spatial layout).

A. Datasets

Malimg. The download link given in [1] no longer works, so we used a public mirror: Kaggle dataset "ikrambenabd/malimg-original", version 1, downloaded on 2 Nov 2026[1]. It contains 9,339 grayscale PNG images in 25 families, 119 fewer than the 9,458 reported in [1]; recent work [9] uses the same 9,339 images. Malimg ships as pre-rendered byteplots, so the width table was not applied by us and the original binaries are not available. Family sizes range from 80 (Skintrim.N) to 2,949 (Allaple.A).

BIG 2015. The Microsoft Malware Classification Challenge [18] released hexadecimal dumps (`.bytes`) and disassembly (`.asm`) of 10,868 labelled training samples in nine families; test labels were never released. The dataset's creators removed the PE headers. We use only the labelled training set and only the `.bytes` files, which we render with the width table of [1]. Bytes the dump marks as unknown (`??`) are dropped rather than mapped to zero, which shifts later byte offsets slightly. Storage limits ruled out extracting the full set, so we drew a random subset of at most 250 samples per family with seed 620, keeping all 42 Simda samples (2,042 samples; the list is committed to the repository). One Ramnit sample consists only of `??` bytes and is excluded, leaving 2,041. Table II summarizes both datasets.

B. Deduplication and Splits

Exact duplicates. We hash each image's pixels with SHA-256. Malimg contains only 8,019 distinct images: 14.9% of all images share their exact pixels with at least one other image, and every one of the 800 Yuner.A images is the same image. BIG 2015, by contrast, contains no exact duplicates in our subset.

Near duplicates. Two samples are near duplicates if the Euclidean distance between their GIST descriptors is below a

[1] https://www.kaggle.com/datasets/ikrambenabd/malimg-original; archive SHA-256 0457c29a1b792704…

TABLE I
PUBLISHED RESULTS AND WHETHER EACH IS COMPARABLE TO OURS. "INDICATIVE" = DIFFERS FROM OUR SETUP IN AT LEAST ONE RESPECT.

Work	Method	Samples	Split	Preprocessing / input	Reported	Comparable?
Malimg						
[1]	GIST + k-NN	9,458	not stated	GIST 320-D; input size not stated	Acc. ≈98%	Indicative (count, split)
[3]	VGG16 (M-CNN)	9,339	stratified 90/10, 1 run	$224 \times 224 \times 3$; VGG16 init	Acc. 98.52%	Yes, vs. E1 (single split)
[4]	3-layer CNN (from scratch)	9,342†	10-fold CV, 1 run	256×256 (Lanczos)	Acc. 98.48%, macro-F1 95.8	Yes, vs. E1 (single run)
[5]	IMCFN (VGG16, color)	9,435‡	70/30, 1 run	224×224 color map; augmentation	Acc. 98.82%, F1 98.75	Indicative (color, augm., split)
[9]	3-layer CNN + SVM-SMOTE*	9,339	80/20, 1 run	64×64; oversampling	Acc. 98.39%	Indicative (oversampling, split)
BIG 2015						
[3]	VGG16 (M-CNN)	10,868	90/10 of train set, 1 run	$224 \times 224 \times 3$	Acc. 98.99%§	Indicative (full set, split)
[4]	3-layer CNN (from scratch)	10,868	10-fold CV, 1 run	128×128 (Lanczos)	Acc. 97.5%, macro-F1 94.0	Indicative (full set, split)
Ours (E1)	GIST + k-NN, ResNet-50, byte hist. + LR	9,339	stratified 10-fold, 5 seeds	as shipped; GIST 64×64, ResNet 224×224	Table IV	–
Ours (E3)	same three models	2,041	grouped 10-fold, 5 seeds	width table of [1]	Table IV	–

*Best of four balancing methods in [9] (SVM-SMOTE 98.39, VAE 96.95, Pro-WaS 96.34, AC-GAN 94.74%; 96.09% without). †Stated in Sec. 4.2; the same paper also gives 9,458. ‡Stated; its per-family table sums to 9,342. §Held-out 10% (Setting-A); the often-quoted 99.97% is accuracy on the training set (Setting-B).

TABLE II
DATASETS AS USED IN THIS WORK.

	Malimg [1]	BIG 2015 [18]
Source	Kaggle "ikrambenabd/malimg-original", version 1	Kaggle competition (train set)
Samples used	9,339 (9,458 in [1])	2,041 of 10,868
Families	25	9
Input format	pre-rendered grayscale PNG	`.bytes` hex dumps (no PE headers)
Rendering	none (as shipped)	width table of [1]; `??` dropped
Sampling	all	≤250 per family (seed 620); all 42 Simda
Used in	E1, E2, E4	E3

threshold τ; groups are closed transitively (union-find). We set τ from the distribution of each image's distance to its nearest neighbour, not from classification accuracy. The distribution has a spike of copies near zero, a valley around 0.07, and a mode at the typical same-family distance of about 0.12; every nearest neighbour closer than 0.12 belongs to the same family. We chose $\tau = 0.05$, below the valley. Visual inspection of nearest-neighbour pairs confirms that pairs below about 0.08 are near copies, while pairs around 0.12 (e.g. within Allaple) share only a layout. Table V shows that the conclusions are stable for τ between 0.03 and 0.09. At $\tau = 0.05$, Malimg's 9,339 images form 6,629 groups and 31.1% of images belong to a group of two or more; BIG 2015's 2,041 samples form 1,922 groups.

Splits. E1 uses stratified 10-fold cross-validation, a standard protocol (the protocol of [1] is not stated; A1). E2 and E3 use stratified grouped 10-fold cross-validation, which keeps every group in a single fold. Every experiment is repeated with five seeds (0–4) that change the fold assignment and model initialization; we report the mean and standard deviation over seeds of the out-of-fold predictions pooled within each seed.

Untestable families. Grouping exposes a problem that a stratified split hides: when all of a family's samples fall into one group, the family is never in the training data while it is tested. At $\tau = 0.05$ this happens to three Malimg families: Yuner.A, Autorun.K and Obfuscator.AD. Autorun.K and Yuner.A even merge into one group, the only group that spans two families; Gibert et al. [4] likewise found the byteplots of these two UPX-packed families indistinguishable, differing only in their resource section. Their recall under E2 is zero by construction, which says nothing about leakage, so we report E2 (and E4, which attacks the E2 models) both over all families and over the 8,291 images of the remaining "testable" families (A25). No BIG 2015 family is affected.

C. Models

GIST + k-NN. We implemented GIST from scratch following [2] with the parameters of [1]: 20 Gabor sub-bands over four scales and a 4×4 grid, giving 320 features. Each image is resized to 64×64, log-transformed, whitened and contrast-normalized, padded, and filtered in the frequency domain; the magnitude of each response is averaged over the grid. The descriptor feeds a 3-nearest-neighbour classifier with Euclidean distance and no feature scaling. Nataraj et al. leave several choices unstated: the split, k, the input size and how the 20 sub-bands divide among the four scales, and they refer to Oliva and Torralba [2] for GIST's internal constants. Table III

separates what [1] states from what we assumed. An exact reproduction is therefore impossible, which is itself a finding in the sense of [8].

ResNet-50. An ImageNet-pretrained ResNet-50 [10] with a new final layer is fine-tuned end to end. Byteplots are resized to 224×224 with antialiased bilinear interpolation, replicated to three channels and normalized with ImageNet statistics. We train for 5 epochs with AdamW (learning rate 10^{-4}, weight decay 10^{-4}, batch size 64), mixed precision and a class-weighted cross-entropy loss to counter family imbalance. We use no data augmentation, so that every model sees the same images; some published CNN results on Malimg use augmentation (Table I).

Byte histogram + LR. The normalized 256-bin histogram of byte values, standardized on the training fold, feeds a multinomial logistic regression ($C = 1$). It discards all spatial structure and serves as a test of how easy the benchmark is.

D. Threat Model and Attacks

The attacker controls a malware sample and wants it assigned to the wrong family (untargeted evasion) without changing its behaviour. The only manipulation allowed is appending bytes to the end of the file, which preserves functionality for PE files [11], [16]. The budget is the number of appended bytes as a fraction of the original size: 1, 2, 5 and 10%. After appending, the file is re-rendered with the width table, so the image may change width as well as height; for Malimg, which has no binaries, we treat the image's pixels as the file's bytes (A20). We attack the E2 models on their own test folds.

Three black-box attacks need no knowledge of the model: *random* appends uniform random bytes, *zero* appends zero bytes, and *other family* appends a contiguous run of bytes taken at a random offset from a training sample of a different family. One white-box attack targets the CNN: *FGSM* starts from appended zeros and takes one step of size ϵ along the sign of the loss gradient, applied only to the appended pixels, with the gradient flowing through the CNN's resize and normalization (A22, A23). We chose ϵ on a separate pilot model, not on the evaluation models. On a ResNet-18 trained on 3,000 Malimg images, $\epsilon = 0.05$ was the strongest step, reducing accuracy at a 5% budget to 0.35 against 0.73 for plain zero appending, while $\epsilon = 1.0$ overshoots and reaches only 0.90. The pilot model is separate from the evaluation models but uses images from the same dataset.

E. Metrics and Statistics

We report accuracy and macro-averaged precision, recall, F1 and one-vs-rest ROC-AUC, so that each family counts equally regardless of size, plus per-family recall. Paired differences between models on the same predictions are tested with an exact McNemar test for each seed. For E4 we report accuracy and evasion rate (the share of correctly classified test samples that the attack makes misclassified) at each budget, and summarize each curve by a robustness AUC: the area under accuracy versus budget, normalized so that 100 means no degradation. All experiments ran on lovegood.cs.rit.edu, a server with 10 NVIDIA Tesla T4 GPUs (16 GB each) and 48 CPU cores, one experiment and seed per GPU; the complete run took 2 h 12 min of wall-clock time (12.9 GPU-hours).

V. Results

A. RQ1: Reproduction

Table IV (E1) answers RQ1. GIST + k-NN reaches 97.46% accuracy, within about half a point of the ≈98% reported in [1], with a very small spread across seeds (±0.04). Given that [1] used 119 more images and does not state its split, k or input size, we consider the reproduction successful.

Accuracy hides a weakness that [1], which reports only accuracy, could not show. Its macro-F1 is 93.56%, because recall collapses on a few closely related families: 38.3% for Swizzor.gen!I, 50.8% for Swizzor.gen!E, 72.6% for C2LOP.P and 91.9% for C2LOP.gen!g, which are mostly confused with their sibling variants. All other families exceed 97%. Accuracy is dominated by the large, easy families (Allaple.A alone is 32% of Malimg).

The fine-tuned ResNet-50 reaches 99.29 ± 0.04% accuracy and 98.15 ± 0.11% macro-F1, in line with or above the 98.52–98.82% reported for fine-tuned CNNs [3], [5], and it lifts the hard Swizzor and C2LOP families to 82–98% recall. The most telling result, however, is the weak baseline: a byte histogram with logistic regression, which knows nothing about where bytes occur, reaches 96.82 ± 0.09% accuracy, less than a point below GIST + k-NN, and a *higher* macro-F1 (97.29 ± 0.09%). All pairwise differences are significant in every seed (McNemar, $p \leq 0.017$). On Malimg under a stratified split, a texture descriptor is barely needed: much of the family signal is in the byte distribution itself.

B. RQ2: Duplicates and Leakage

Over all families, the grouped split looks devastating: GIST + k-NN falls from 97.46% to 84.29 ± 0.04%, the CNN to 87.91 ± 0.09% and the byte histogram to 84.93 ± 0.05%. Most of this drop, however, comes from the three untestable families, which are 11% of Malimg and score zero by construction. The testable-family columns (marked † in Table IV) isolate the effect of duplicates: GIST + k-NN loses 2.51 points of accuracy (to 94.95 ± 0.05%) and 6.91 points of macro-F1 (to 86.65 ± 0.12%), the byte histogram 1.15 and 1.62 points, and the CNN only 0.26 and 0.64. Leakage therefore inflates the nearest-neighbour method far more than the CNN, which fits how each works: k-NN succeeds whenever a near copy of the test image is in the training set, whereas the CNN learns features that transfer to unseen variants.

Table VI shows where this happens. The clearest case is Dontovo.A: 99.4% of its images sit in duplicate groups, and GIST + k-NN recall falls from 100% to 0.6%, while the CNN keeps 100%. Wintrim.BX (100% to 89.7%) and Lolyda.AA2 (97.3% to 92.3%) follow the same pattern, whereas families without duplicates, including the hard Swizzor and C2LOP variants, barely change. Table V shows why near-duplicate

TABLE III
REPRODUCTION PARAMETERS: STATED IN [1] VS. ASSUMED (An).

	Parameter	Value used	Source / reason
	Byteplot width table	by file size in KB	stated [1]
	GIST scales / orientations	4 / 8	stated [1]
	GIST sub-bands N	20	stated [1]
	Spatial grid $M \times M$	4×4	stated [1]
	Feature length	320 ($= M \cdot M \cdot N$)	stated [1]
	Classifier / distance	k-NN / Euclidean	stated [1]
A1	Folds	stratified 10-fold, 5 seeds	not stated; standard k-fold protocol
A2	GIST input size	64×64	not stated; keeps GIST cheap (not varied)
A3	k	3	not stated; small odd k avoids two-way ties
A4	Orientations per scale	8, 4, 4, 4 (fine→coarse)	total of 20 stated; keeps the stated 8 at the finest scale
A5	Kilobyte	1,024 bytes	[1] writes "kB" without defining it; 1,000 B would widen 2.4% of BIG 2015 images
A6	Width-table bounds	lower bound inclusive	ranges share endpoints; matches "<10 kB" (no file lies on a bound)
A7	Averaging window w	block mean, 4×4	window not specified
A8	Partial last row	zero-padded	not stated
A9	Filter centre frequencies	0.3 cycles/px, $\div 1.85$ per scale	reference GIST code [2]
A10	Radial bandwidth	Gaussian, $\sigma = 0.2$ on $r/f_c - 1$	our choice (see text)
A11	Angular bandwidth	Gaussian, $\sigma = (\pi/n)/2.355$	our choice: neighbours cross at half max.
A12	Orientation range	one-sided, 0–180°, 0° = horizontal freq.	reference GIST code [2]
A13	Intensity transform	scale to 0–255, then $\log(x+1)$	reference GIST code [2]
A14	Border padding	5 px, symmetric (mirror)	reference GIST code [2]
A15	Whitening	Gaussian low-pass, cut-off 4 cycles/image, subtracted	reference GIST code [2]
A16	Local contrast normalisation	same Gaussian, divide by $0.2 + \sigma_{\text{local}}$	reference GIST code [2]
A17	Spatial pooling	non-overlapping 4×4 block means, row-major	reference GIST code [2]
A18	Padding before Gabor filtering	32 px, symmetric, cropped after	reference GIST code [2]
A19	Filter response	magnitude $\lvert \cdot \rvert$ of the complex output	reference GIST code [2]
A20	Append budget unit	byteplot pixels, incl. zero-padded last row (A8)	Malimg has no binaries, only images
A21	Donor bytes (other-family)	contiguous run at random offset; repeated if too short	avoids always copying a header
A22	FGSM	single step, untargeted, cross-entropy, $\epsilon = 0.05$; appended bytes only	ϵ from a separate pilot (see text)
A23	Attacker knowledge	white-box, gradient through resize and normalisation	worst case for the defender
A24	Near-duplicate threshold	GIST distance < 0.05, merged transitively	below the NN-distance valley (Table V)
A25	Single-group families	excluded from E2/E4 headline metrics; all-family numbers also reported	never in training when tested

TABLE IV
MEAN ± STD OVER SEEDS (OUT-OF-FOLD, POOLED PER SEED).

Model	E1		E2†		E2 (all fam.)		E3	
	Acc. (%)	Macro-F1 (%)	Acc. (%)	Macro-F1 (%)	Acc. (%)	Macro-F1 (%)	Acc. (%)	Macro-F1 (%)
GIST + k-NN [1]	97.46 ± 0.04	93.56 ± 0.09	94.95 ± 0.05	86.65 ± 0.12	84.29 ± 0.04	73.63 ± 0.13	90.43 ± 0.19	88.02 ± 0.20
ResNet (fine-tuned)	99.29 ± 0.04	98.15 ± 0.11	99.03 ± 0.10	97.51 ± 0.34	87.91 ± 0.09	82.72 ± 0.22	94.04 ± 0.21	93.77 ± 0.46
Byte histogram + LR	96.82 ± 0.09	97.29 ± 0.09	95.67 ± 0.06	95.67 ± 0.17	84.93 ± 0.05	80.67 ± 0.15	87.06 ± 0.21	84.92 ± 0.28

†Excluded (single duplicate group, never in training when tested): Autorun.K, Obfuscator.AD, Yuner.A.

grouping matters: with exact duplicates alone, testable accuracy is 97.21%, indistinguishable from E1, because the leaked pairs are near copies rather than identical files. From $\tau = 0.03$ to 0.09 testable accuracy stays between 94.91% and 95.13%, so the conclusion does not depend on the exact threshold.

C. RQ3: Cross-Dataset

On BIG 2015 the ranking from Malimg holds, but every model loses ground: the CNN reaches $94.04 \pm 0.21\%$ accuracy ($93.77 \pm 0.46\%$ macro-F1), GIST + k-NN $90.43 \pm 0.19\%$ ($88.02 \pm 0.20\%$), and the byte histogram $87.06 \pm 0.21\%$ ($84.92 \pm 0.28\%$). The CNN's lead over GIST + k-NN (3.61 points) is similar to the one under the grouped split on Malimg (4.08), but the byte histogram, which matched GIST + k-NN on Malimg, now falls 3.37 points behind it: spatial texture carries more information on BIG 2015 than on Malimg. The weakest families differ by model: GIST + k-NN struggles most with Simda (62.9%, only 42 samples) and Obfuscator.ACY (75.7%), the byte histogram with Lollipop (58.6%), and the CNN, unusually, with Kelihos_ver3 (88.3%), where the other two models exceed 99%. These numbers are indicative only when compared with published BIG 2015 results such as 98.99% [3]: those use the full, imbalanced training set and different splits, while our capped subset is more balanced and our split is grouped (Table I).

TABLE V
SENSITIVITY TO THE NEAR-DUPLICATE THRESHOLD (A24), MALIMG.

Threshold	Groups	In a group (%)	Single-group families	Acc. all (%)	Acc. testable (%)
exact only	8019	14.9	Yuner.A	88.89	97.21
0.01	7096	25.8	Autorun.K, Yuner.A	86.02	95.26
0.03	6765	29.4	Autorun.K, Obfuscator.AD, Yuner.A	84.46	95.13
0.05*	6629	31.1	Autorun.K, Obfuscator.AD, Yuner.A	84.29	94.95
0.07	6399	33.6	Autorun.K, Obfuscator.AD, Yuner.A	84.34	95.00
0.09	6250	35.1	Autorun.K, Obfuscator.AD, Yuner.A	84.26	94.91

GIST + k-NN, grouped 10-fold split, mean over 5 seeds. *Value used.

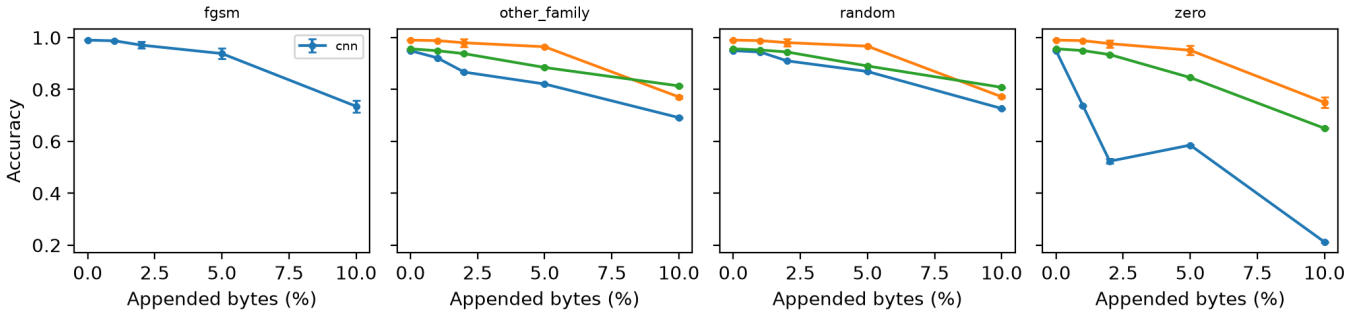


Fig. 1. Accuracy vs. append budget on the E2 test folds, testable families (mean ± std over seeds).

D. RQ4: Robustness

Figure 1 and Table VII answer RQ4. GIST + k-NN is remarkably fragile to the cheapest possible attack. Appending zeros worth 1% of the file lowers its accuracy from 94.95% to 73.79% and at 10% to 21.23% (robustness AUC 51.3), with 78.7% of previously correct samples evaded. Random and other-family bytes are much weaker (AUC 85.3 and 81.4). A plausible explanation is that a block of zeros adds a large uniform region, which changes the low-frequency content that GIST summarizes far more than noise does. The zero-attack curve is not monotonic (52.35% at 2% but 58.55% at 5%); we did not isolate the cause, but re-rendering can change the image width and the resize to 64×64 changes the aspect ratio, so the effect of a budget depends on the file size. The CNN is far more robust: it loses less than two points up to a 2% budget and stays above 93.85% at 5% under every attack, but all attacks bring it to 73.53–77.19% at 10%. Because random, zero and other-family bytes have almost the same effect at 10%, the drop seems to come from the change in image geometry rather than from the appended content. White-box FGSM is only slightly stronger than the black-box attacks (AUC 90.2 versus 91.2–92.4), which is consistent with its single small step (A22); an iterative attack would be stronger. The byte histogram falls between the two (AUC 83.1 for zeros, 89.0 and 88.8 for random and other-family bytes): appended bytes shift its histogram directly, but in proportion to the budget.

VI. DISCUSSION

Is Malimg still a useful benchmark? Our results suggest that it is useful for checking that a pipeline works, but not for ranking methods. A 256-bin histogram nearly matches a texture descriptor; one family is a single image repeated 800 times and two others collapse into single groups once near duplicates are merged; and the headline accuracy of the nearest-neighbour method owes part of its value to near copies shared between training and test folds. These are the sampling-bias and data-snooping pitfalls described by Arp et al. [8], and they apply to every paper in Table I that uses a random split.

Why do GIST and the CNN converge on Malimg? Under a stratified split both approaches can rely on memorizing near copies, so the benchmark compresses their differences into the last two points of accuracy. Remove that crutch (E2) or change the dataset (E3), and the gap roughly doubles: under the grouped split the CNN keeps its accuracy while GIST + k-NN loses most on precisely the families with many near duplicates. Macro-F1 and per-family recall expose the gap that accuracy hides, and we recommend reporting them.

Robustness and deployment. A classifier that appending 1% zero bytes can defeat is not suitable for deployment against an adversary, however accurate it is on clean data. The CNN's robustness is better but not complete: at a 10% budget roughly a quarter of correct predictions are evaded by any attack. Because appending is free for an attacker and preserves functionality, image-based classifiers should be evaluated against it by default.

TABLE VI
PER-FAMILY DUPLICATE RATE AND RECALL, E1 (RANDOM) VS. E2 (GROUPED).

Family	n	Dup. (%)	Recall E1	Recall E2
Adialer.C	122	99.2	100.0	100.0
Agent.FYI	116	0.0	100.0	99.3
Allaple.A	2949	0.7	99.9	99.9
Allaple.L	1591	0.0	99.2	99.1
Alueron.gen!J	198	0.0	99.0	99.0
Autorun.K‡	106	100.0	100.0	0.0
C2LOP.P	146	0.0	72.6	72.1
C2LOP.gen!g	200	0.0	91.9	92.0
Dialplatform.B	177	7.9	98.9	98.9
Dontovo.A	162	99.4	100.0	0.6
Fakerean	381	97.6	99.9	99.9
Instantaccess	431	99.8	100.0	100.0
Lolyda.AA1	213	64.3	99.0	98.2
Lolyda.AA2	184	94.0	97.3	92.3
Lolyda.AA3	123	96.7	98.4	98.4
Lolyda.AT	159	22.6	100.0	100.0
Malex.gen!J	136	0.0	98.5	98.5
Obfuscator.AD‡	142	100.0	100.0	0.0
Rbot!gen	158	65.8	98.7	98.7
Skintrim.N	80	97.5	100.0	100.0
Swizzor.gen!E	128	0.0	50.8	53.0
Swizzor.gen!I	132	0.0	38.3	37.9
VB.AT	408	2.5	99.2	99.2
Wintrim.BX	97	85.6	100.0	89.7
Yuner.A‡	800	100.0	100.0	0.0

‡One duplicate group: never in training when tested (E2).

TABLE VII
ROBUSTNESS AUC (%), AREA UNDER ACCURACY-VS-BUDGET; 100 = NO DEGRADATION.

Model	Random	Zero pad	Other family	FGSM (white-box)
GIST + k-NN [1]	85.3 ± 0.1	51.3 ± 0.2	81.4 ± 0.1	–
ResNet (fine-tuned)	92.4 ± 0.3	91.2 ± 1.2	92.3 ± 0.3	90.2 ± 1.4
Byte histogram + LR	89.0 ± 0.1	83.1 ± 0.1	88.8 ± 0.2	–

VII. LIMITATIONS AND THREATS TO VALIDITY

Data provenance. We could not obtain Malimg from its original source, and we cannot confirm that the mirror we used is identical to the authors' data; its image count already differs (9,339 versus 9,458). For BIG 2015, storage limits, not a methodological choice, forced us to use a 2,041-sample subset rather than the full 10,868-sample training set; our BIG 2015 results are therefore indicative only.

Reproduction choices. Several parameters of [1] are unstated, and our assumptions (Table III) may differ from the original. Even the width table is open to interpretation: it writes "kB" without defining it, and reading it as 1,000 rather than 1,024 bytes would render 2.4% of our BIG 2015 images one width larger (A5). We did not vary the GIST input size, k or the division of sub-bands among scales, so we cannot rule out that another choice would close the remaining half-point gap.

Threat model. Malimg provides images, not binaries, so we treat image pixels as file bytes and cannot verify that a perturbed file still runs. Our FGSM is a single step chosen on a pilot model that shares the evaluation dataset; it is a lower bound on what a white-box attacker can do.

Temporal and family coverage. Neither dataset has timestamps, so we cannot test the temporal bias that TESSERACT [7] identifies, and 25 and 9 families are far fewer than a vendor sees.

Compute. A from-scratch run needs 12.9 GPU-hours, so the repository's one-command check re-merges committed per-seed results instead of recomputing them (Appendix A).

VIII. CONCLUSION AND FUTURE WORK

We reproduced the GIST + k-NN malware-image classifier of Nataraj et al. to within about half a point of its reported accuracy (RQ1), but found that the benchmark it introduced overstates what the method can do. Duplicate-aware splitting removes 6.9 points of macro-F1 from GIST + k-NN on the families that can be tested, against 0.6 for a fine-tuned ResNet-50 (RQ2); the ranking holds on BIG 2015, where all models lose accuracy (RQ3); and a 1% append of zero bytes evades about a quarter of GIST + k-NN's correct predictions (RQ4). Future work should test stronger, iterative and mask-guided attacks [9], [17], include binaries so that functionality can be verified, and evaluate on time-stamped datasets with temporal splits [7].

REFERENCES

[1] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, "Malware images: Visualization and automatic classification," in *Proc. 8th Int. Symp. Visualization for Cyber Security (VizSec)*, Pittsburgh, PA, USA, 2011, art. no. 4.

[2] A. Oliva and A. Torralba, "Modeling the shape of the scene: A holistic representation of the spatial envelope," *Int. J. Comput. Vis.*, vol. 42, no. 3, pp. 145–175, 2001.

[3] M. Kalash, M. Rochan, N. Mohammed, N. D. B. Bruce, Y. Wang, and F. Iqbal, "Malware classification with deep convolutional neural networks," in *Proc. 9th IFIP Int. Conf. New Technologies, Mobility and Security (NTMS)*, 2018, pp. 1–5.

[4] D. Gibert, C. Mateu, J. Planes, and R. Vicens, "Using convolutional neural networks for classification of malware represented as images," *J. Comput. Virol. Hacking Tech.*, vol. 15, no. 1, pp. 15–28, 2019.

[5] D. Vasan, M. Alazab, S. Wassan, H. Naeem, B. Safaei, and Q. Zheng, "IMCFN: Image-based malware classification using fine-tuned convolutional neural network architecture," *Comput. Netw.*, vol. 171, 2020, art. no. 107138.

[6] D. Vasan, M. Alazab, S. Wassan, B. Safaei, and Q. Zheng, "Image-based malware classification using ensemble of CNN architectures (IMCEC)," *Comput. Secur.*, vol. 92, 2020, art. no. 101748.

[7] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Security Symp.*, 2019, pp. 729–746.

[8] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 3971–3988.

[9] M. A. Tariq, A. B. Sargano, Z. Habib, and X. Huang, "An advanced framework for malware detection using adversarial learning," *Int. J. Inf. Secur.*, vol. 25, 2026, art. no. 139.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[11] L. Demetrio, S. E. Coull, B. Biggio, G. Lagorio, A. Armando, and F. Roli, "Adversarial EXEmples: A survey and experimental evaluation of practical attacks on machine learning for Windows malware detection," *ACM Trans. Privacy Secur.*, vol. 24, no. 4, pp. 1–31, 2021.

[12] L. Nataraj, V. Yegneswaran, P. Porras, and J. Zhang, "A comparative assessment of malware classification using binary texture analysis and dynamic analysis," in *Proc. 4th ACM Workshop on Security and Artificial Intelligence (AISec)*, 2011, pp. 21–30.
[13] S. Ni, Q. Qian, and R. Zhang, "Malware identification using visualization images and deep learning," *Comput. Secur.*, vol. 77, pp. 871–885, 2018.
[14] M. Brosolo, V. Puthuvath, A. K. A., R. Rehiman, and M. Conti, "SoK: Visualization-based malware detection techniques," in *Proc. 19th Int. Conf. Availability, Reliability and Security (ARES)*, 2024, pp. 1–13.
[15] E. Raff, J. Barker, J. Sylvester, R. Brandon, B. Catanzaro, and C. Nicholas, "Malware detection by eating a whole EXE," in *Workshops of the 32nd AAAI Conf. Artificial Intelligence*, 2018, pp. 268–276.
[16] B. Kolosnjaji, A. Demontis, B. Biggio, D. Maiorca, G. Giacinto, C. Eckert, and F. Roli, "Adversarial malware binaries: Evading deep learning for malware detection in executables," in *Proc. 26th European Signal Processing Conf. (EUSIPCO)*, 2018, pp. 533–537.
[17] D. Zhan, Y. Duan, Y. Hu, L. Yin, Z. Pan, and S. Guo, "AMGmal: Adaptive mask-guided adversarial attack against malware detection with minimal perturbation," *Comput. Secur.*, vol. 127, 2023, art. no. 103103.
[18] R. Ronen, M. Radu, C. Feuerstein, E. Yom-Tov, and M. Ahmadi, "Microsoft malware classification challenge," arXiv:1802.10135, 2018, kaggle competition dataset (BIG 2015).

APPENDIX A
CODE AND REPRODUCIBILITY

All code, configurations and per-seed results are available at https://github.com/Fussyness/csec-620-research-paper-432004751. Every number and table in this paper is generated from the configuration `config.yaml` (5 seeds $\times$ 10 folds). `make reproduce` needs no datasets and finishes in seconds: it reuses the committed per-seed results, each stamped with a fingerprint of the configuration and the commit it was produced from, and regenerates `results/metrics.json` and every table. `make reproduce-scratch` recomputes everything from the datasets (about 13 h on one T4 GPU), and a provided script runs the same jobs in parallel on a multi-GPU server. The datasets must be downloaded manually as described in `data/README.md`; they cannot be redistributed.

APPENDIX B
AI USE AND STATEMENT OF ORIGINAL WORK

This project used Claude (Anthropic) as an assistant. It helped find and verify references and plan the experiments (Milestone 2); scaffolded the experiment pipeline (data loading, deduplication and folds, model wrappers, evaluation, tables, tests and configurations), the parallel-run scripts, the FGSM step-size pilot and the near-duplicate threshold analysis; and drafted the text of this paper from the author's results. The GIST descriptor and the append and FGSM attacks were written by the author, with step-by-step guidance and review from the assistant. The author chose the research questions, ran the experiments, checked every number against the results, and made sure the data extracted from the test run and the data taken from accompanying papers were correctly placed before finalizing the full draft.